Practica 1: Analisis de Componentes Principales

Rafael Ortega de la Paz — No. de cuenta 420054085

Facultad de Ingenieria, Universidad Nacional Autonoma de Mexico — Reconocimiento de Patrones

Resumen—Este reporte documenta la implementacion y aplicacion del analisis de componentes principales (PCA) sobre cuatro conjuntos de datos: un dataset sintetico bivariado, Iris, Wine y Olivetti Faces. Se implementaron desde cero dos rutas de calculo equivalentes —mediante la matriz de covarianza (`pca_cov`) y mediante descomposicion en valores singulares (`pca_svd`)— y se compararon contra la implementacion de scikit-learn, confirmando su equivalencia numerica. Se analizan criterios de seleccion del numero de componentes (varianza acumulada, regla del codo, criterio de Kaiser), el efecto de estandarizar los datos, y la relacion entre el numero de componentes retenidas y el error de reconstruccion, ilustrada con el caso de EigenFaces.

I. Dataset sintético de dos variables

Se genero un conjunto sintetico de n=200 muestras bivariadas, con $x_1 \sim N(0,1)$ y $x_2 = 2x_1 + N(0,1.25)$, de forma que ambas variables estan fuertemente correlacionadas por construccion. Los datos se centraron restando la media de cada columna y, sobre los datos centrados, se calculo la matriz de covarianza junto con sus valores y vectores propios mediante `numpy.linalg.eigh`, ordenando el resultado de mayor a menor varianza (PC1 = mayor varianza).

```
Xc = X - X.mean(axis=0)
cov = np.cov(Xc, rowvar=False)
eigvals, eigvecs = np.linalg.eigh(cov)
orden = np.argsort(eigvals)[::-1]
eigvals, eigvecs = eigvals[orden], eigvecs[:, orden]
```

La media original fue [0.0709, -0.0182] y, tras centrar, se redujo a valores del orden de 10^{-17} (numericamente cero). La matriz de covarianza resultante fue:

```
Cov = [[1.0485, 2.1886],
       [2.1886, 5.9575]]
```

Los valores propios obtenidos fueron λ_1=6.7915 y λ_2=0.2145, con vectores propios asociados v_1=(0.3561, 0.9344) y v_2=(-0.9344, 0.3561). La razon $\lambda_1/\lambda_2 \approx 31.7$ confirma que casi toda la varianza del conjunto se concentra en una sola direccion, consistente con el hecho de que x_2 se construyo como una funcion (casi) lineal de x_1. El producto punto entre v_1 y v_2 fue de 7.48×10^{-18} y la matriz V^TV resulto practicamente identidad, por lo que los vectores propios son ortogonales entre si, como garantiza la teoria para una matriz de covarianza (simetrica).

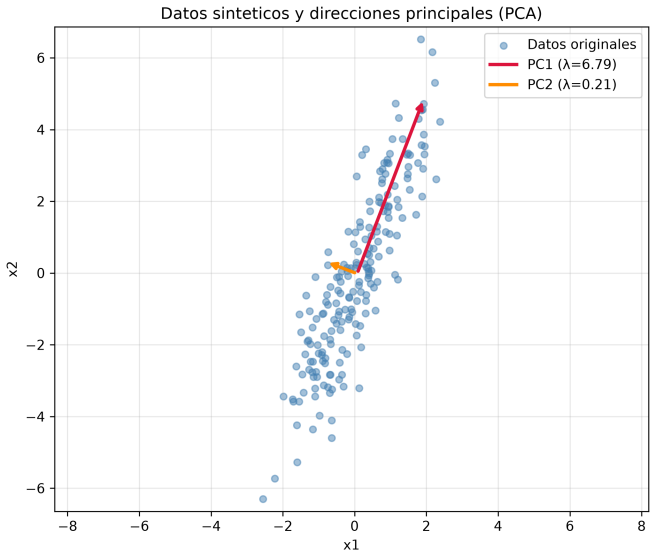


Fig. 1. Datos sinteticos (n=200) y direcciones principales. Cada vector propio se grafico escalado por la raiz de su valor propio, partiendo de la media de los datos.

La Fig. 1 muestra que PC1 se alinea con la direccion de maxima dispersion de la nube de puntos (la diagonal principal), mientras que PC2, perpendicular a PC1, apunta en la direccion de mucha menor varianza, coherente con los valores propios calculados.

II. Dataset Iris

Sobre las 150 muestras y 4 caracteristicas (largo/ancho de sepalo y petalo) del dataset Iris se genero primero un `pairplot` con seaborn, coloreado por especie.

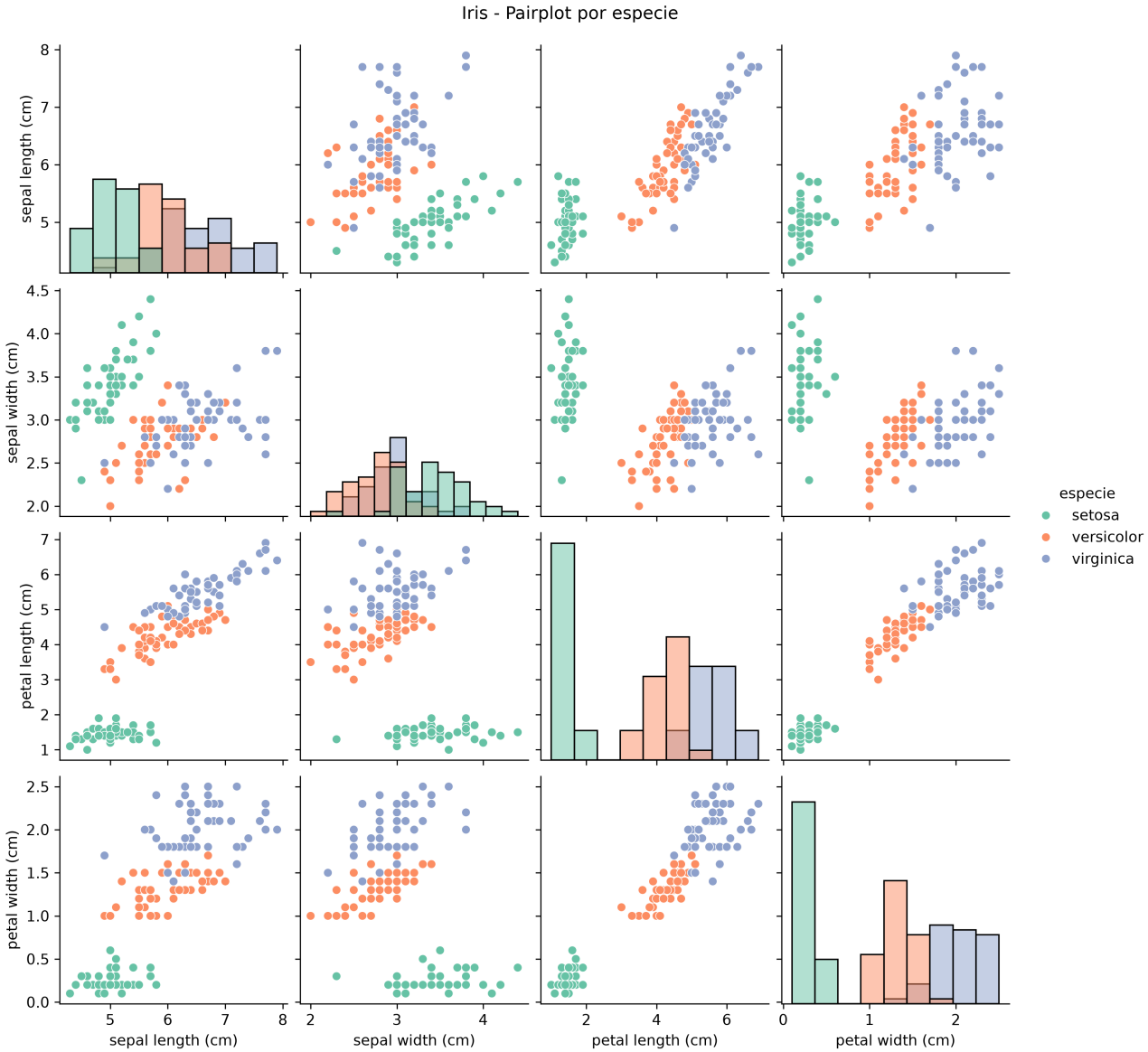


Fig. 2. Pairplot del dataset Iris, coloreado por especie (setosa, versicolor, virginica).

Como se observa en la Fig. 2, los pares que incluyen largo y ancho de petalo (petal length / petal width) son los que mejor separan las tres clases: setosa forma un grupo completamente aislado, mientras que versicolor y virginica, aunque cercanas, muestran una frontera razonablemente clara en esas dos variables. Los pares que usan unicamente medidas de sepalo separan mucho peor a versicolor de virginica.

Se implemento la funcion `pca_cov(X, k)`, que centra los datos, calcula la matriz de covarianza, obtiene sus valores/vectores propios y regresa los datos proyectados Z junto con los k vectores y valores propios de mayor varianza:

```
def pca_cov(X, k):
    Xc = X - X.mean(axis=0)
    cov = np.cov(Xc, rowvar=False)
    eigvals, eigvecs = np.linalg.eigh(cov)
    orden = np.argsort(eigvals)[::-1]
    eigvals, eigvecs = eigvals[orden], eigvecs[:, orden]
    eigvals_k, eigvecs_k = eigvals[:k], eigvecs[:, :k]
    Z = Xc @ eigvecs_k
    return Z, eigvecs_k, eigvals_k
```

Aplicando `pca_cov` con k=4 se obtuvieron los valores propios λ=(4.2282, 0.2427, 0.0782, 0.0238), con varianza explicada de 92.46%, 5.31%, 1.71% y 0.52% respectivamente (acumulada: 92.46%, 97.77%, 99.48%, 100%).

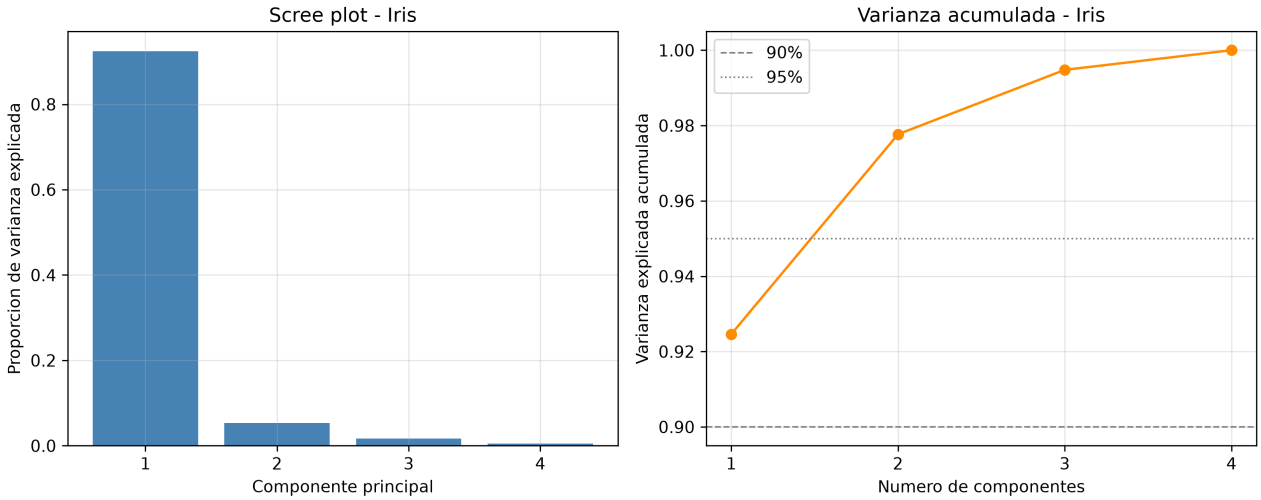


Fig. 3. Scree plot (izquierda) y varianza explicada acumulada (derecha) para Iris, con lineas de referencia en 90% y 95%.

La Fig. 3 muestra que la primera componente por si sola ya explica 92.46% de la varianza total, es decir, cae dentro del rango de 90%–95% solicitado; por lo tanto, con un solo vector propio (el asociado a λ_1=4.2282, con carga principal en petal length y petal width) bastaria para cumplir ese criterio. Si se buscara superar el 95%, se necesitarian 2 componentes (97.77% acumulado).

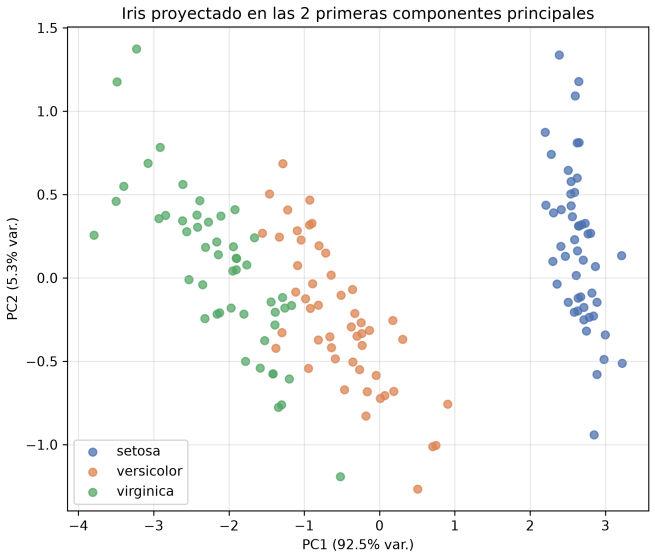


Fig. 4. Proyeccion de Iris sobre sus dos primeras componentes principales, coloreada por especie.

En la Fig. 4, al proyectar los datos sobre PC1-PC2, la especie setosa queda completamente separada del resto a lo largo de PC1, mientras que versicolor y virginica muestran un ligero traslape, en linea con lo observado en el pairplot original. Es importante notar que PCA no elimina ni crea caracteristicas fisicas: construye NUEVAS variables (PC1, PC2, ...) que son combinaciones lineales de las 4 caracteristicas originales, elegidas para maximizar la varianza capturada; la informacion de las variables originales no se descarta, se re-expresa de forma comprimida en menos dimensiones.

III. Dataset Wine

Se implemento la funcion `pca_svd(X, k)`, que entrega la misma informacion que `pca_cov` pero calculada mediante SVD de los datos centrados X_c=USV^T: los valores propios de la covarianza equivalen a $S^2/(n-1)$ y los vectores propios son las columnas de V.

```
def pca_svd(X, k):
    Xc = X - X.mean(axis=0)
    n = Xc.shape[0]
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    eigvals = (S**2) / (n - 1)
    eigvecs = Vt.T
    eigvals_k, eigvecs_k = eigvals[:k], eigvecs[:, :k]
    Z = Xc @ eigvecs_k
    return Z, eigvecs_k, eigvals_k
```

El dataset Wine (178 muestras, 13 caracteristicas fisico-quimicas) se estandarizo (media 0, desviacion estandar 1 por columna) y se aplico `pca_svd` con k=13.

Autovalores y varianza explicada (datos estandarizados):

PC	λ	Var. expl.	Var. acum.
1	4.7324	36.20%	36.20%
2	2.5111	19.21%	55.41%
3	1.4542	11.12%	66.53%
4	0.9242	7.07%	73.60%
5	0.8580	6.56%	80.16%
6	0.6453	4.94%	85.10%
7	0.5541	4.24%	89.34%
8	0.3505	2.68%	92.02%
9	0.2905	2.22%	94.24%
10	0.2523	1.93%	96.17%
11	0.2271	1.74%	97.91%
12	0.1697	1.30%	99.20%
13	0.1040	0.80%	100.00%

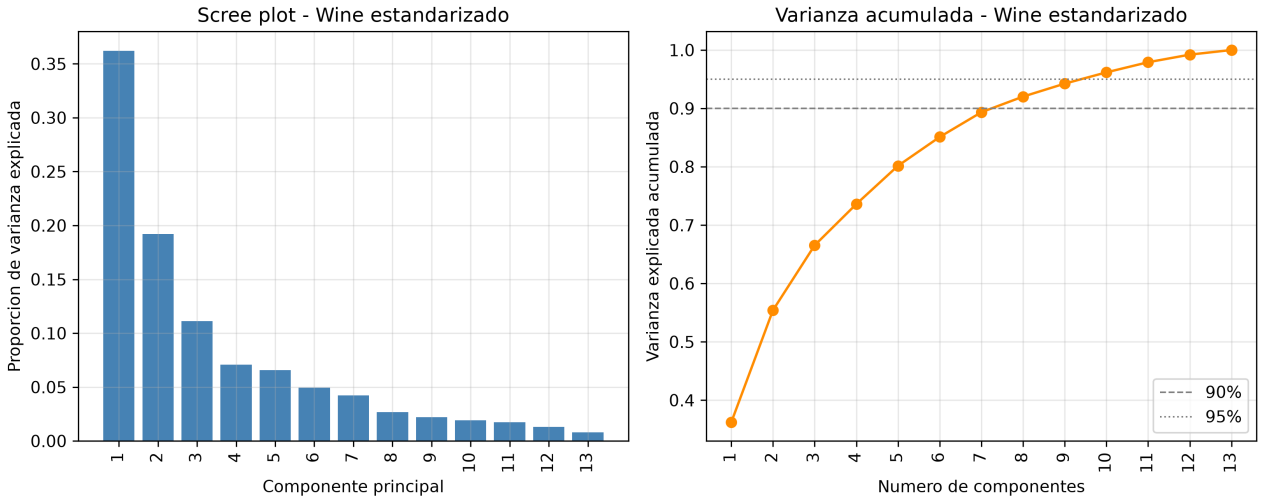


Fig. 5. Scree plot y varianza explicada acumulada para Wine, datos estandarizados.

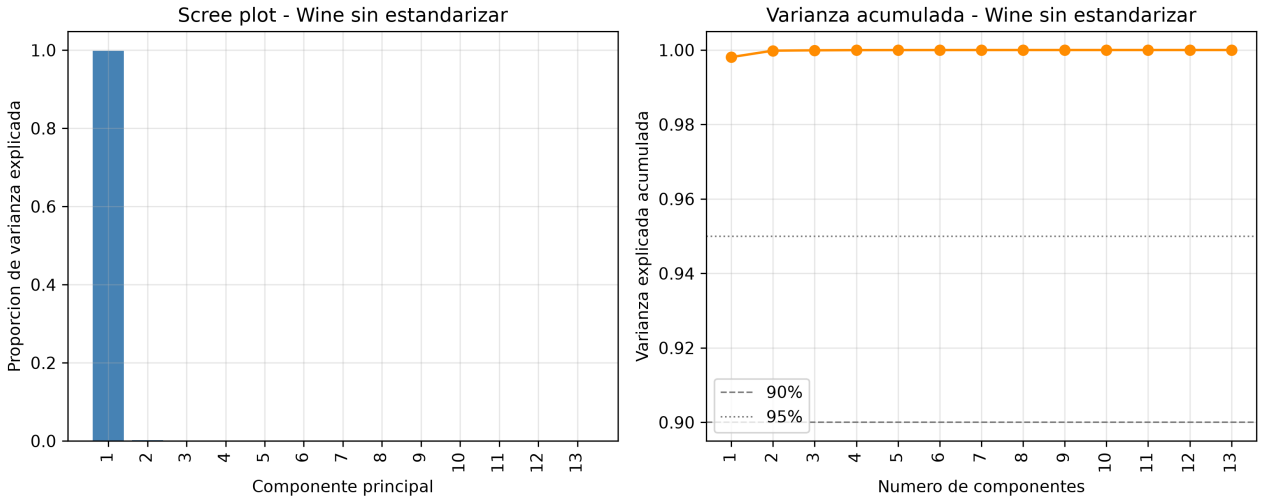


Fig. 6. Scree plot y varianza explicada acumulada para Wine, datos SIN estandarizar.

Con base en la Fig. 5 y la tabla anterior, los tres criterios de decision vistos en clase dan resultados distintos. (i) *Umbral de varianza*: en clase se establecio como meta estandar capturar entre 90% y 95% de la varianza; aqui se requieren 8 componentes para superar 90% y 10 para superar 95%. (ii) *Criterio de Kaiser*: definido en clase como retener los componentes con autovalor >1 (validos sobre la matriz de correlacion, es decir, datos estandarizados); sugiere 3 componentes (λ=4.73, 2.51, 1.45). (iii) *Regla del codo*: en clase se definio de forma visual y heuristica —sin una formula rigurosa— como el punto donde ocurre el cambio mas abrupto en la curva y este empieza a aplanarse; en la Fig. 5 ese aplanamiento se observa claramente entre los componentes 3 y 4 (caida de 1.45 a 0.92, seguida de caidas mucho menores). Como aproximacion numerica reproducible de esa inspeccion visual (calculando la distancia perpendicular maxima entre la curva y la cuerda que une su primer y ultimo punto) se obtiene el componente 4, consistente con la lectura visual de la grafica. Los tres criterios son razonables y coinciden en un rango compacto (3–4 componentes) si se prioriza compresion, o hasta 8–10 si se prioriza preservar casi toda la varianza.

Al aplicar PCA sobre los datos sin estandarizar (Fig. 6), la primera componente concentra 99.81% de la varianza total (contra 36.20% en el caso estandarizado), y con solo 2 componentes se alcanza 99.98%. Esto ocurre porque la variable 'proline' tiene una escala numerica (cientos a miles) muchisimo mayor que el resto (p. ej. 'ash', en el rango 1-3), por lo que domina artificialmente la varianza total y, con ello, casi toda la primera componente. El efecto de no estandarizar es que PCA deja de reflejar la estructura de correlacion real entre variables y en su lugar refleja principalmente diferencias de escala/unidades, lo cual es indeseable cuando las variables se miden en unidades distintas (como es el caso de Wine).

Finalmente, se reconstruyeron los datos estandarizados usando k=1,…,13 componentes ($X_{rec}=Z_k V_k^T+\mu$) y se calculo el MSE entre X y su reconstruccion:

k	MSE
1	0.6380
2	0.4459
3	0.3347
4	0.2640
5	0.1984
6	0.1490
7	0.1066

k	MSE
8	0.0798
9	0.0576
10	0.0383
11	0.0209
12	0.0080
13	0.0000

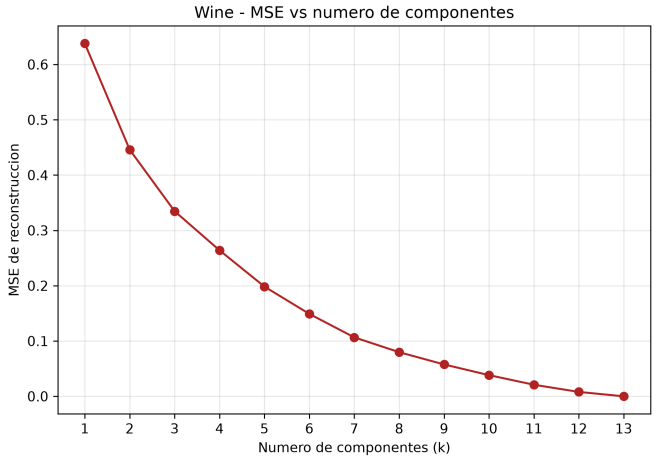


Fig. 7. MSE de reconstruccion vs. numero de componentes k, dataset Wine (estandarizado).

La Fig. 7 confirma que el MSE decrece monotonamente y de forma cada vez mas lenta, llegando a 0 en k=13 (reconstruccion exacta al usar todas las componentes, ya que no se pierde informacion).

IV. Comparación de métodos (Iris)

Se aplico PCA sobre Iris con tres implementaciones distintas: `pca_cov`, `pca_svd` y `sklearn.decomposition.PCA`, comparando los 4 valores y vectores propios que entrega cada una.

Valor propio	pca_cov	pca_svd	sklearn.PCA
λ1	4.22824171	4.22824171	4.22824171
λ2	0.24267075	0.24267075	0.24267075
λ3	0.07820950	0.07820950	0.07820950
λ4	0.02383509	0.02383509	0.02383509

Los cuatro valores propios coinciden exactamente (diferencias ≤ 3.55×10^{-15}, error de punto flotante) entre las tres implementaciones. Para los vectores propios, las tres tecnicas entregan las mismas direcciones, pero antes de alinear signos algunas columnas difieren por un factor de -1 (p. ej. la columna 1 de `pca_cov` es exactamente la columna 1 de `pca_svd` multiplicada por -1). Tras fijar el signo de cada vector con respecto a una referencia comun, la diferencia maxima absoluta entre vectores propios de cualquier par de metodos fue de 2.11×10^{-15}.

¿A que se debe esta diferencia? Ni los eigenvectores de una matriz simetrica ni los vectores singulares de una SVD tienen un signo unico definido: si v es un vector propio (o vector singular) unitario, -v tambien lo es, y ambos representan la misma direccion/eje principal. Las tres implementaciones son matematicamente equivalentes —diagonalizar la matriz de covarianza (`pca_cov`) y aplicar SVD a los datos centrados (`pca_svd`, y lo que `sklearn.PCA` hace internamente) resuelven el mismo problema de optimizacion— por lo que las magnitudes coinciden salvo el signo, que depende de detalles de implementacion del algoritmo numerico subyacente (LAPACK) y no tiene significado estadistico.

V. EigenFaces (Olivetti Faces)

El dataset Olivetti Faces (400 imagenes de rostro de 64×64 pixeles, 40 sujetos) se cargo ya en forma de matriz X de (400, 4096) —n=400 imagenes, p=4096 pixeles por imagen— y se centro restando el rostro promedio (media de X). Sobre X centrada se aplico `pca_svd`, reutilizando la misma funcion de la Seccion III, obteniendo hasta 400 componentes (eigenfaces).



Fig. 8. Rostro promedio del dataset Olivetti Faces (media de X).

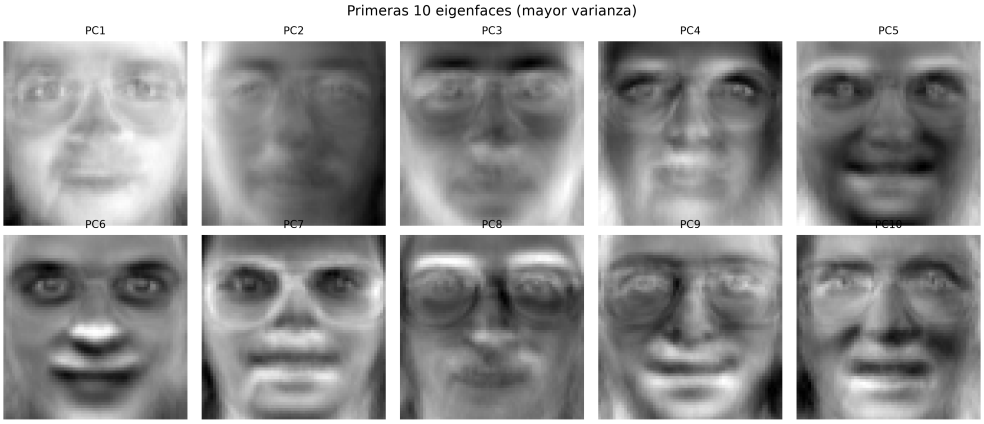


Fig. 9. Primeras 10 eigenfaces (mayor varianza).

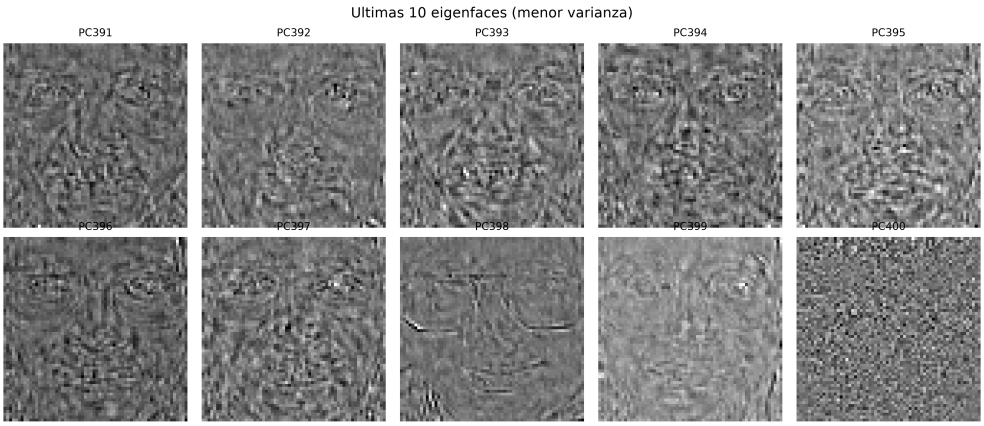


Fig. 10. Ultimas 10 eigenfaces (menor varianza).

La Fig. 8 muestra el rostro promedio, un rostro suave y generico producto de promediar las 400 imagenes. Las Fig. 9 y 10 muestran, respectivamente, las eigenfaces de mayor y menor varianza. Las primeras capturan patrones globales y de baja frecuencia que varian de forma consistente entre muchos sujetos (iluminacion general, contorno de la cara, presencia de lentes, sombras alrededor de ojos/nariz/boca); son reconocibles como rostros 'borrosos' pero estructurados. Las ultimas eigenfaces, en cambio, lucen como patrones de alta frecuencia, ruidosos y sin estructura facial clara: capturan variaciones muy especificas de pocas imagenes (posiblemente ruido de adquisicion), consistente con sus valores propios cercanos a cero.

Se reconstruyo una imagen de prueba utilizando k=10, 50, 100, 200 y 300 eigenfaces, comparando contra la imagen original:



Fig. 11. Reconstruccion de una imagen de Olivetti Faces usando k=10,50,100,200,300 eigenfaces, junto con la imagen original.

En la Fig. 11 se aprecia como, con pocos componentes (k=10), la reconstruccion es muy borrosa y solo conserva rasgos generales; conforme k crece, se recuperan progresivamente detalles finos (lentes, textura de barba, arrugas) hasta lucir practicamente identica a la original con k=300 de un total de 400 posibles.

Se calculo tambien el MSE de reconstruccion sobre todo el dataset, variando k en intervalos de 20:

k	MSE
20	0.004559
60	0.002093
100	0.001246
140	0.000791
180	0.000507
220	0.000319
260	0.000193
300	0.000106
340	0.000047
380	0.000010
400	0.000000

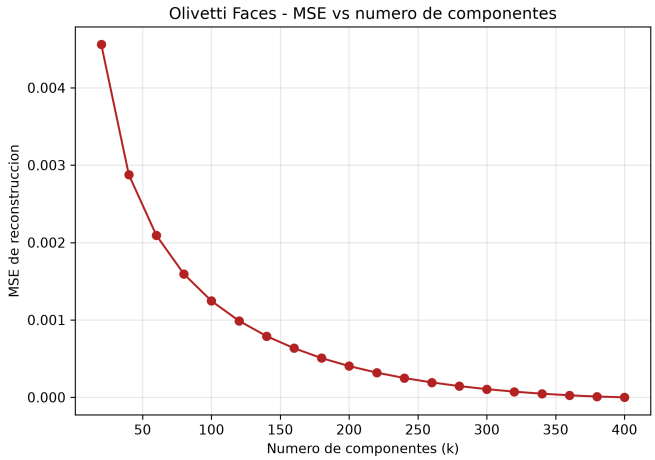


Fig. 12. MSE de reconstruccion vs. numero de eigenfaces utilizadas k.

Como muestra la Fig. 12, el error decrece de forma monotona y convexa. En k=300, la varianza acumulada explicada es de 99.45% (0.55% de varianza descartada), lo que es consistente con un MSE ya muy cercano a cero (0.000106) en ese punto. Esto responde directamente a la pregunta de interpretacion: el error de reconstruccion disminuye al aumentar k porque cada eigenface adicional que se incorpora aporta exactamente la varianza asociada a su valor propio, informacion que antes se estaba descartando. De hecho, el MSE de reconstruccion es, salvo un factor de escala (1/p, con p el numero de pixeles), la suma de los valores propios de las componentes NO utilizadas (la varianza descartada): al ordenar las componentes de mayor a menor varianza, cada eigenface adicional agregada es la que mas reduce ese residuo en cada paso, por lo que el MSE cae rapido al inicio (las primeras eigenfaces concentran la mayor parte de la varianza) y se aplana conforme se agregan componentes cuyo valor propio es cada vez mas pequeno.

VI. Conclusiones

El desarrollo de esta practica permitio verificar de forma empirica varias propiedades teoricas de PCA: los vectores propios de una matriz de covarianza son ortogonales entre si; el calculo via matriz de covarianza y via SVD son equivalentes (salvo signo) y coinciden con la implementacion de referencia de scikit-learn; estandarizar es indispensable cuando las variables tienen escalas distintas, pues de lo contrario PCA queda dominado por la variable de mayor varianza numerica en lugar de reflejar la estructura de correlacion real de los datos; y el error de reconstruccion al truncar a k componentes esta directamente ligado a la varianza (valores propios) descartada en las componentes no retenidas, lo cual se observo tanto en Wine como en EigenFaces.